

Trabajo Práctico 6 — Menú, HUD y Game Over

Diplomatura de Videojuegos · Clase 6 Objetivo: convertir el mini-juego del TP5 en un **juego completo, con principio y final**. Vas a armar un **menú principal**, ampliar el **HUD** para mostrar las bombas atrapadas, y agregar una **condición de derrota** que lleva a una pantalla de **Game Over** con el puntaje final.

Qué vas a lograr

Un juego que ya se puede entregar de punta a punta:

- Arranca en un **menú** con el título y los botones **Jugar** y **Salir**.
- El **HUD** muestra dos datos en vivo: **puntos** y **bombas atrapadas**.
- A la **tercera bomba** el juego termina solo y salta a **Game Over**.
- La pantalla de Game Over muestra el **puntaje final** y permite **Reintentar** o volver al **Menú**.

 **Tiempo estimado:** 45–70 min. Seguí sobre el **mismo proyecto del TP5**: no empezás de cero.

 **Viene de la Clase 6:** nodos `Control`, `CanvasLayer`, `Label`, `Button` con su señal `pressed()`, y `change_scene_to_file()` para navegar entre escenas.

Punto de partida

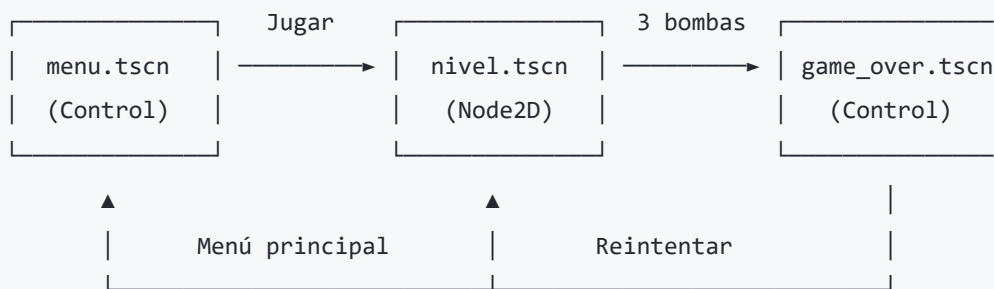
Este TP continúa el proyecto `tp5-atrapar`. Abrilo (o **hacé una copia** de la carpeta y llamala `tp6-menu`, así conservás el TP5 entregado).

Vas a seguir usando `nivel.tscn`, `Canasta`, `Spawner` y `HUD` tal como quedaron. **No hace falta tocar** `objeto_cae.gd`, `comida.gd`, `basura.gd` ni `spawner.gd`: toda la herencia del TP5 queda intacta.

 **Recordá los nombres del TP5:** la clase base es `objeto_cae.gd`; `comida.gd` son las **piezas de Tetris** (suman +1) y `basura.gd` son las **bombas** (restan -1). Los nombres de archivo están en genérico a propósito, para que el sistema sirva para cualquier tema.

Las tres escenas del juego

Hoy el juego pasa de tener **una** escena a tener **tres**, y navegás entre ellas:



Las tres se conectan con **la misma función**: `get_tree().change_scene_to_file("res://...")`.

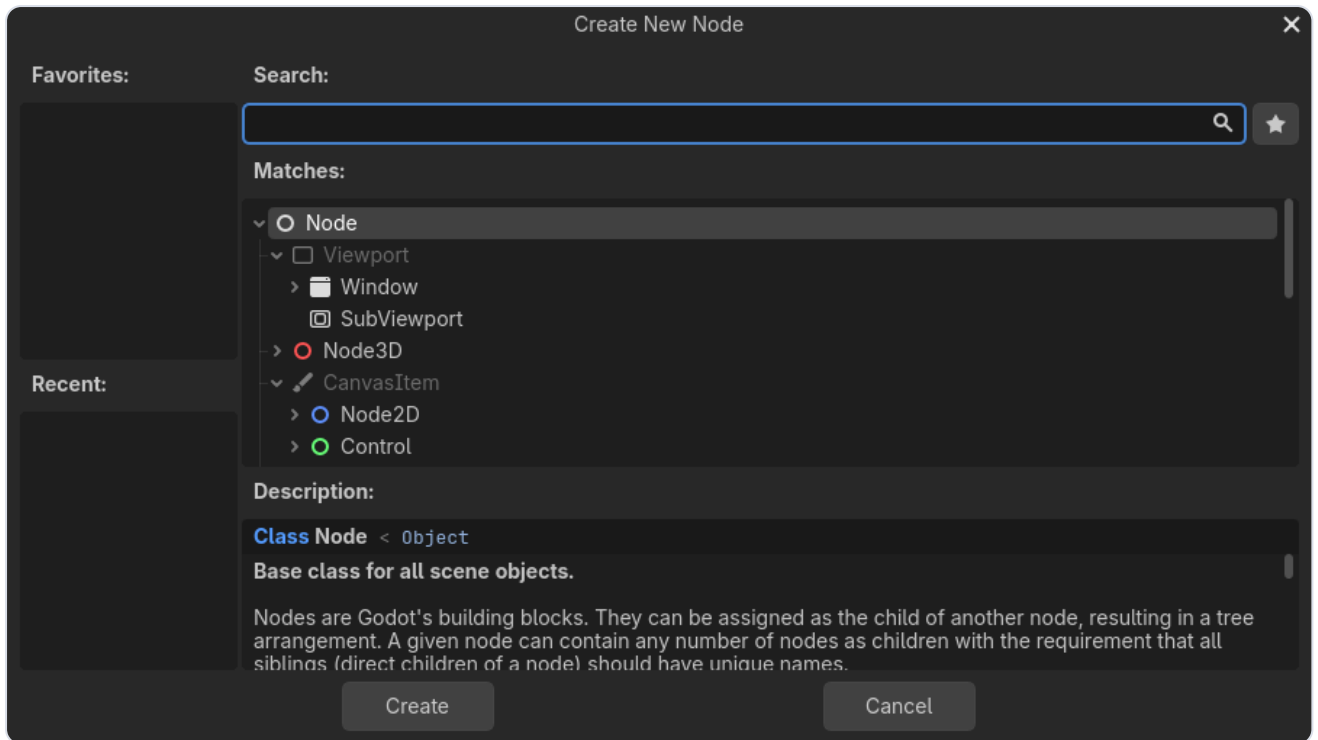
Parte 0 — Retomar el proyecto

1. Abrí el proyecto del TP5 (o la copia `tp6-menu`).
 2. Ejecutá `nivel.tscn` con **F6** y confirmá que el **punto de control 6 del TP5** sigue andando: el `Label` del HUD sube con las piezas y baja con las bombas.
- ✓ **Punto de control 0:** el juego del TP5 corre y el puntaje se actualiza en pantalla.

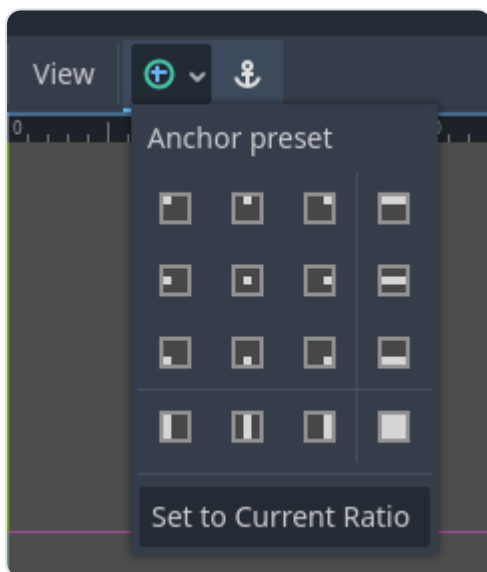
Parte 1 — La escena del menú

Concepto: los menús se arman con nodos `Control`, igual que el HUD del TP5 — pero acá ocupan **toda la pantalla**, en vez de ser una capa encima del juego.

1. **Escena** → **Otro Nodo** → `Control` → renombralo `MenuPrincipal`. Guardá con **Ctrl+S** como `menu.tscn`.



- Hijo de `MenuPrincipal` (**Ctrl+A**) → `ColorRect` → renómbrolo `Fondo`. Con `Fondo` seleccionado, en la barra del Viewport abrí el menú de **anclas** y elegí **Full Rect**: así el fondo cubre toda la pantalla, sea cual sea la resolución.



🗨 **Las anclas (anchors)** le dicen a un nodo `Control` a qué parte de la pantalla se pega. `Full Rect` = "ocupá todo". `Center` = "quedate en el centro". Es lo que hace que la UI **sobreviva a cualquier resolución**, en vez de quedar clavada a unos píxeles fijos.

- Hijo de `MenuPrincipal` → `VBoxContainer` → renómbrolo `Botonera`. Ancla → **Center**. (Opcional) Para que no quede todo pegado: **Inspector** → **Theme Overrides** → **Constants** → **Separation** = `20`.
- Hijo de `Botonera` → `Label` → renómbrolo `LabelTitulo`. Propiedad **Text** = `Atrapa las piezas`. Agrandá la letra en **Theme Overrides** → **Font Sizes** → **Font Size** = `48`.
- Hijo de `Botonera` → `Button` → renómbrolo `BtnJugar`. **Text** = `Jugar`.

6. Hijo de `Botonera` → `Button` → renombralo `BtnSalir`. `Text` = `Salir`.

El árbol de la escena

```
MenuPrincipal (Control)
├─ Fondo      (ColorRect)      ← ancla Full Rect
└─ Botonera   (VBoxContainer)  ← ancla Center
    ├─ LabelTitulo (Label)
    ├─ BtnJugar   (Button)
    └─ BtnSalir   (Button)
```

🗨️ **¿Por qué un `VBoxContainer`?** Porque **acomoda a sus hijos solo**, uno debajo del otro, sin que tengas que posicionar nada a mano. Agregás un botón más y se reordena todo automáticamente.

✅ **Punto de control 1:** al ejecutar `menu.tscn` con **F6**, ves el título y los dos botones centrados en pantalla.

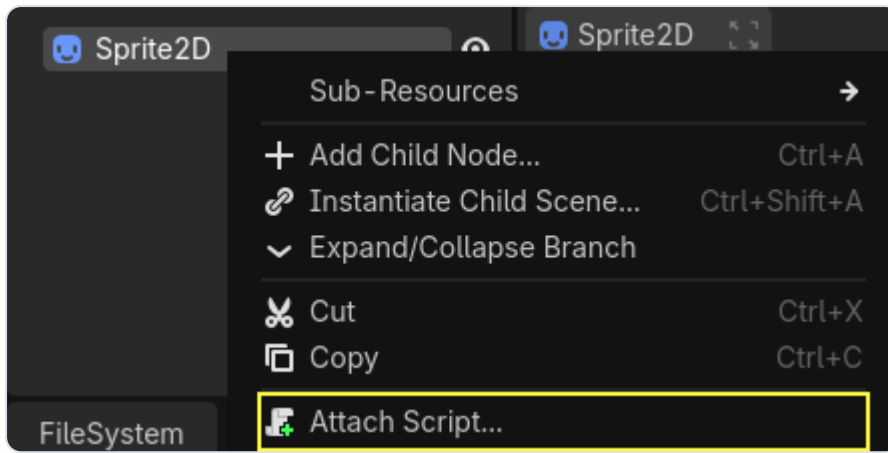
🚨 Los botones quedan en una esquina, o el fondo no cubre todo

▼ Abrí para ver soluciones

- El `Fondo` necesita el ancla **Full Rect**; la `Botonera`, **Center**. El menú de anclas aparece en la barra de arriba del Viewport solo cuando tenés seleccionado un nodo `Control`.
- Si los botones se ven deformados, revisá que sean hijos del `VBoxContainer` y no del `Control` raíz.
- ¿El `ColorRect` tapa los botones? Tiene que estar **arriba** de la `Botonera` en el árbol: en 2D, **lo de más abajo se dibuja encima**.

🧠 Parte 2 — El código del menú

1. Clic derecho en `MenuPrincipal` → **Attach Script** → dejá `res://menu.gd` → **Create**.



2. Escribí:

```
extends Control

@onready var boton_jugar = $Botonera/BtnJugar
@onready var boton_salir = $Botonera/BtnSalir

func _ready() -> void:
    boton_jugar.pressed.connect(_on_jugar_pressed)
    boton_salir.pressed.connect(_on_salir_pressed)

func _on_jugar_pressed() -> void:
    get_tree().change_scene_to_file("res://nivel.tscn")

func _on_salir_pressed() -> void:
    get_tree().quit()
```

🗨 Qué está pasando acá:

- `pressed` es la señal que emite un `Button` al hacer clic. La conectamos a nuestra función igual que hicimos con `body_entered` en el TP4.
- `change_scene_to_file()` descarga el menú **por completo** y carga el nivel. No hace falta `queue_free()` ni limpiar nada a mano.
- `$Botonera/BtnJugar` es la ruta del nodo dentro de la escena: primero el contenedor, después el botón.

✅ **Punto de control 2:** apretás **Jugar** y arranca el nivel del TP5. Apretás **Salir** y se cierra la ventana.

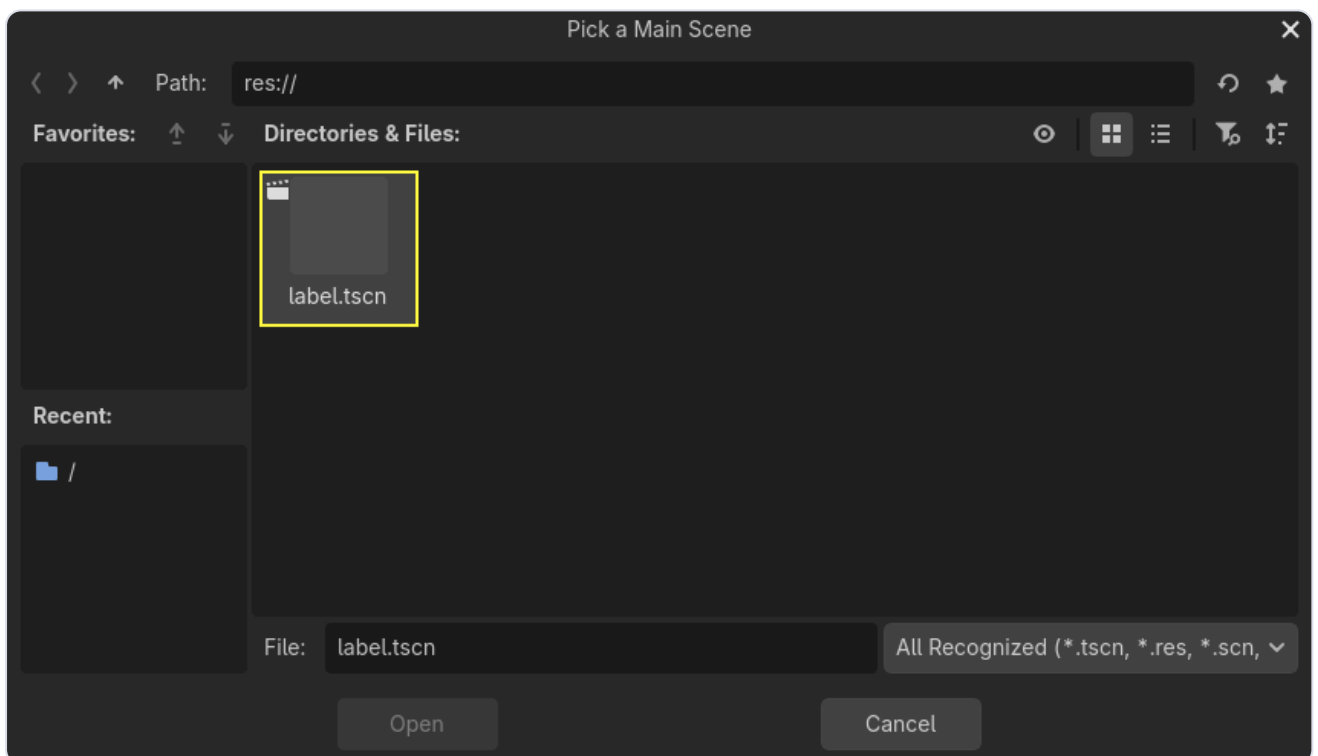
🚫 No pasa nada al hacer clic, o tira error de nodo

▼ Abrí para ver soluciones

- Las rutas de `@onready` tienen que coincidir **exactamente** con tu árbol: si el contenedor se llama distinto, cambiá `$Botonera/BtnJugar`.
- Si dice que no encuentra `res://nivel.tscn`, fijate el nombre real en el **FileSystem**: ¿es `nivel.tscn` o `Nivel.tscn`? Tiene que coincidir **con mayúsculas y todo**.
- **Salir** cierra la ventana del juego: probalo con **F5**, no desde el editor.

❏ Parte 3 — Que el juego arranque en el menú

1. Project → Project Settings → General → Application → Run → Main Scene: elegí `menu.tscn`.



2. Apretá **F5** (ejecutar el **proyecto**, no la escena).

💬 **F5 vs F6:** `F6` corre **la escena que tenés abierta**; `F5` corre **el juego de verdad**, empezando por la *Main Scene*. De ahora en más, F5 es la forma correcta de probar el juego completo.

- ✓ **Punto de control 3:** **F5** abre el menú, y desde ahí **Jugar** te lleva al nivel.

✂ Parte 4 — Ampliar el HUD: contador de bombas

Concepto: en el TP5 el HUD mostraba solo el puntaje. Ahora sumamos un segundo `Label` dentro del mismo `CanvasLayer` — el patrón de la Clase 6.

1. Abrió `nivel.tscn`.
2. Seleccioná el nodo `HUD` → **Ctrl+A** → `Label` → renombralo `LabelBombas`.
3. Propiedad **Text** = `Bombas: 0`. Ubicalo **debajo** de `LabelPuntos`.

El árbol del HUD, ahora

```
HUD (CanvasLayer)
├── LabelPuntos
└── LabelBombas ← nuevo
```

⚠ **No metas los Labels dentro de un contenedor todavía.** Es tentador agregar un `VBoxContainer` para alinearlos, pero eso **cambia la ruta** de los nodos: `../HUD/LabelPuntos` pasaría a ser `../HUD/VBoxContainer/LabelPuntos` y el código del TP5 dejaría de encontrarlos. Si querés hacerlo igual, acordate de **actualizar las dos rutas** en `canasta.gd`.

✅ **Punto de control 4:** en el editor ves los dos `Label` en pantalla (el de bombas todavía dice `0` fijo).

🔗 Parte 5 — Contar las bombas en `canasta.gd`

Concepto clave: `canasta.gd` ya recibe `sumar_puntos(valor)` cada vez que atrapa algo — las piezas mandan `+1` y las bombas `-1`, heredado de `objeto_cae.gd`. **No hace falta tocar ninguna de esas clases:** alcanza con mirar **el signo** de `valor` para saber qué atrapó.

Reemplazá `canasta.gd` por esta versión:

```

extends CharacterBody2D

var velocidad := 500.0
var puntos := 0
var bombas_atrapadas := 0

func _ready() -> void:
    add_to_group("canasta")
    actualizar_hud()

func _physics_process(delta: float) -> void:
    velocity.x = Input.get_axis("ui_left", "ui_right") * velocidad
    move_and_slide()
    var ancho := get_viewport_rect().size.x
    position.x = clamp(position.x, 0, ancho)

func sumar_puntos(valor: int) -> void:
    puntos += valor
    if valor < 0:
        bombas_atrapadas += 1
    actualizar_hud()

func actualizar_hud() -> void:
    get_node("../HUD/LabelPuntos").text = "Puntos: " + str(puntos)
    get_node("../HUD/LabelBombas").text = "🔴 Bombas: " + str(bombas_atrapadas)

```

🗨️ Dos detalles:

- La función `actualizar_puntos()` del TP5 ahora se llama `actualizar_hud()` , porque actualiza **dos** carteles. Es solo un cambio de nombre.
- `if valor < 0` identifica cualquier cosa que reste, **sin preguntar de qué clase es**. Si mañana agregás una pieza dorada de +5 o un enemigo de -3 (el *extra* del TP5), **esta lógica sigue funcionando sin tocarla**. Eso es aprovechar bien la herencia.

✅ **Punto de control 5:** jugás, y **los dos contadores se actualizan en vivo**: el puntaje sube y baja, y el de bombas sube cada vez que atrapás una.

"Node not found: ../HUD/LabelBombas"

▼ Abrió para ver soluciones

- El `Label` tiene que llamarse **exactamente** `LabelBombas` y ser **hijo directo** del `HUD`.
- `Canasta` y `HUD` tienen que ser **hermanos** (los dos, hijos de `Nivel`): por eso la ruta empieza con `../`.
- Si agregaste un contenedor, la ruta cambió: mirá la advertencia de la Parte 4.

Parte 6 — Guardar el puntaje: el Autoload `Partida`

El problema: cuando cambiás de escena, Godot **descarga la anterior por completo**. La variable `puntos` de la canasta **desaparece con ella**. Entonces... ¿cómo hace la pantalla de Game Over para mostrar el puntaje final?

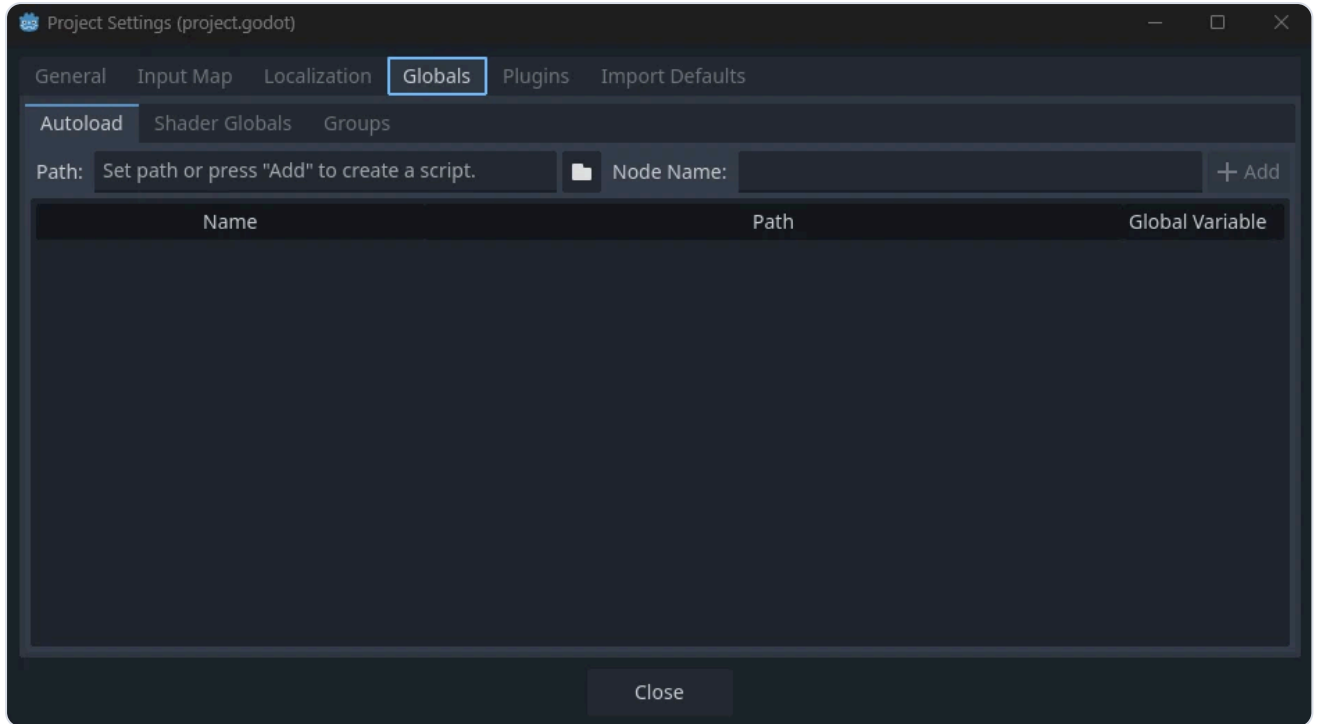
La solución: un **Autoload** (o *singleton*): un script que Godot mantiene **siempre cargado**, sea cual sea la escena activa.

1. En el panel **FileSystem**, clic derecho → **New** → **Script....** Dejá **Inherits:** `Node` y el path en `res://partida.gd` → **Create**.
2. Escribí adentro:

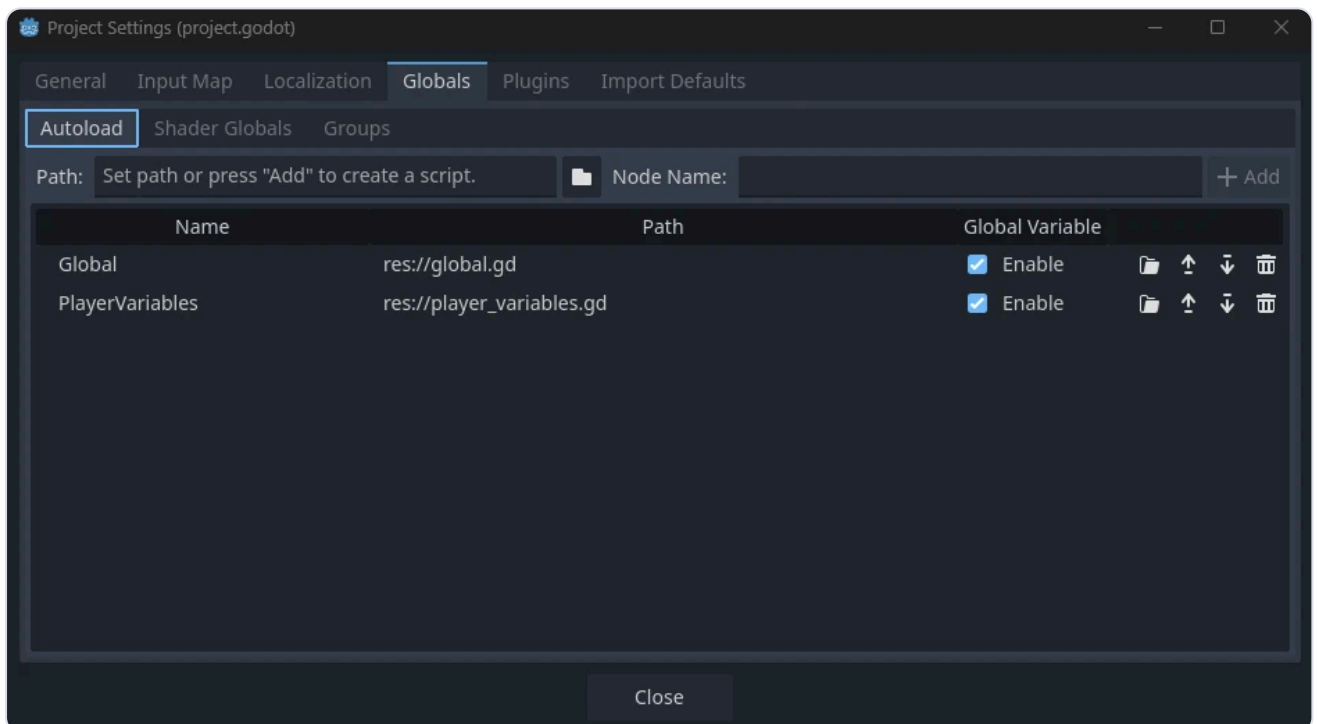
```
extends Node

var ultimo_puntaje := 0
```

3. **Project** → **Project Settings** → **Globals** → **Autoload**.
4. En **Path** elegí `partida.gd`; en **Node Name** escribí `Partida` → **Add**.



5. Tiene que quedar en la lista, con **Global Variable** tildado:



🗨️ **Qué ganás con esto:** `Partida` pasa a existir como **un nodo más del árbol, siempre presente**. Desde cualquier script escribís `Partida.ultimo_puntaje` como si fuera una variable global — sin `@onready`, sin `$`, sin buscarlo. Es la forma estándar en Godot de compartir datos entre escenas.

✅ **Punto de control 6:** en **Project Settings** → **Globals** → **Autoload** aparece `Partida` en la lista, tildado.

No encuentro la pestaña "Autoload"

▼ Abrí para ver soluciones

- En **Godot 4.x** vive en **Project Settings** → **Globals** → **Autoload**. En Godot 3 era una pestaña suelta llamada **AutoLoad**: si seguís un tutorial viejo, es el mismo lugar con otro nombre.
- El **Node Name** (`Partida`) es el nombre con el que lo usás en el código. Si le pusiste otro, usá ese en vez de `Partida` .

Parte 7 — La pantalla de Game Over

1. Escena → Otro Nodo → `Control` → renombralo `GameOver` . Guardá como `game_over.tscn` .
2. Hijo → `ColorRect` → `Fondo` → ancla **Full Rect**. Poné un color oscuro o rojizo (propiedad **Color** en el Inspector) para que se distinga del menú.
3. Hijo de `GameOver` → `VBoxContainer` → `Botonera` → ancla **Center**.
4. Hijo de `Botonera` → `Label` → `LabelTitulo` . **Text** = `Game Over` . Fuente grande.
5. Hijo de `Botonera` → `Label` → `LabelPuntaje` . **Text** = `Puntos: 0` (provisorio: lo va a pisar el código).
6. Hijo de `Botonera` → `Button` → `BtnReintentar` . **Text** = `Reintentar` .
7. Hijo de `Botonera` → `Button` → `BtnMenu` . **Text** = `Menú principal` .

El árbol de la escena

```
GameOver (Control)
├─ Fondo (ColorRect) ← ancla Full Rect
└─ Botonera (VBoxContainer) ← ancla Center
    ├─ LabelTitulo (Label)
    ├─ LabelPuntaje (Label)
    ├─ BtnReintentar (Button)
    └─ BtnMenu (Button)
```

 Es **la misma estructura del menú**, con un `Label` más. Si te salió el menú, esto lo armás en dos minutos.

✅ **Punto de control 7:** al ejecutar `game_over.tscn` con **F6**, ves el título, el puntaje (todavía en 0) y los dos botones centrados.

Parte 8 — El código del Game Over

1. Clic derecho en `GameOver` → **Attach Script** → `res://game_over.gd`.
2. Escribí:

```
extends Control

@onready var label_puntaje = $Botonera/LabelPuntaje
@onready var boton_reintentar = $Botonera/BtnReintentar
@onready var boton_menu = $Botonera/BtnMenu

func _ready() -> void:
    label_puntaje.text = "Puntos: " + str(Partida.ultimo_puntaje)
    boton_reintentar.pressed.connect(_on_reintentar_pressed)
    boton_menu.pressed.connect(_on_menu_pressed)

func _on_reintentar_pressed() -> void:
    get_tree().change_scene_to_file("res://nivel.tscn")

func _on_menu_pressed() -> void:
    get_tree().change_scene_to_file("res://menu.tscn")
```

 Es **el mismo patrón del menú**: dos botones con `pressed` conectado a `change_scene_to_file()`. Lo único nuevo es la primera línea de `_ready()`: leer `Partida.ultimo_puntaje` en vez de un valor local.

✅ **Punto de control 8:** ejecutá `game_over.tscn` con **F6**. Los botones ya funcionan (te llevan al nivel y al menú) y el puntaje muestra `0`, porque todavía nadie lo escribió.

Parte 9 — La derrota: 3 bombas y se acabó

Ahora que la pantalla de Game Over **ya existe**, conectamos la condición de derrota. Reusamos `bombas_atrapadas`, que ya lleva la cuenta desde la Parte 5.

En `canasta.gd`, modificá `sumar_puntos()` y agregá una función nueva al final:

```
func sumar_puntos(valor: int) -> void:
    puntos += valor
    if valor < 0:
        bombas_atrapadas += 1
    actualizar_hud()
    if bombas_atrapadas >= 3:
        game_over()

func game_over() -> void:
    Partida.ultimo_puntaje = puntos
    get_tree().change_scene_to_file("res://game_over.tscn")
```

🗨 **El orden importa:** primero guardamos el puntaje en `Partida`, **después** cambiamos de escena. Al revés no funcionaría: la canasta ya no existiría para leer sus `puntos`.

Apretá **F5** y jugá completo: menú → nivel → atrapá 3 bombas → Game Over con tu puntaje.

✅ **Punto de control 9 (final):** el juego se recorre entero. En Game Over aparece **el mismo puntaje** que tenías al morir, **Reintentar** reinicia el nivel y **Menú principal** vuelve al inicio.

🔧 Errores comunes en esta parte

▼ Abrí para ver soluciones

- **"Identifier 'Partida' not declared":** falta el Autoload (Parte 6), o le pusiste otro **Node Name**. Revisalo en Project Settings → Globals → Autoload.
- **"Cannot load scene res://game_over.tscn":** el archivo no existe con ese nombre exacto, o lo guardaste en una subcarpeta. Mirá el FileSystem y copió la ruta tal cual.
- **El Game Over muestra 0:** falta la línea `Partida.ultimo_puntaje = puntos`, o quedó **después** del `change_scene_to_file()`.
- **Nunca llega el Game Over:** para testear rápido bajá el umbral a `>= 1`, comprobá que funcione y después volvelo a `3`.



Entrega

Entregá **una** de estas opciones:

1. La **carpeta del proyecto** comprimida en `.zip` (sin la carpeta `.godot/`), o
2. Un **video corto** (o GIF) mostrando el recorrido completo: el menú, apretar **Jugar**, atrapar piezas y bombas viendo los dos contadores actualizarse, y la pantalla de **Game Over** al atrapar la tercera

bomba.

Nombre: `tp6-menu-ApellidoNombre.zip`

✓ Checklist de autoevaluación

- ☐ Existe `menu.tscn` con título y botones **Jugar** / **Salir**.
- ☐ **Jugar** carga `nivel.tscn` con `change_scene_to_file()` ; **Salir** cierra con `quit()` .
- ☐ `menu.tscn` está configurado como **Main Scene** (F5 arranca ahí).
- ☐ El HUD tiene `LabelPuntos` y `LabelBombas` , y los dos se actualizan **en tiempo real**.
- ☐ `canasta.gd` identifica las bombas por el **signo de valor** , sin tocar las clases hijas.
- ☐ Existe el **Autoload** `Partida` con la variable `ultimo_puntaje` .
- ☐ Al llegar a **3 bombas**, el juego cambia solo a `game_over.tscn` .
- ☐ Game Over muestra **el mismo puntaje** con el que terminó la partida.
- ☐ Desde Game Over, **Reintentar** vuelve al nivel y **Menú principal** al menú.

☀ Extra (opcional)

- **Botón “Menú” durante el juego:** agregá un `Button` dentro del `HUD` , en una esquina que no moleste, con un script en el `HUD` que llame a `change_scene_to_file("res://menu.tscn")` . Ojo: al salir se pierde la partida en curso.
- **High score:** sumá `var mejor_puntaje := 0` al Autoload `Partida` , actualízalo en `game_over()` si `puntos` lo supera, y mostralo en el menú principal. Es el primer dato que **sobrevive entre partidas**.
- **Vidas visuales:** en vez de un número, mostrá **3 corazones** en el HUD que se apagan con cada bomba.
- **Pausa:** un botón que haga `get_tree().paused = true` y muestre un panel, **sin cambiar de escena**.
- **Transiciones y sonido:** animá la entrada del Game Over con un `Tween` , y sumá un `AudioStreamPlayer` distinto para pieza, bomba y derrota (Clase 7).

Recursos

- Nodos de interfaz: [docs.godotengine.org/es/4.x — Interfaz de usuario \(UI\)](https://docs.godotengine.org/es/4.x/—Interfaz%20de%20usuario%20(UI))
- Contenedores: [GUI containers](#)
- Anclas y tamaños: [Size and anchors](#)
- Cambiar de escena por código: [Change scenes manually](#)
- Autoloads / singletons: [Singletons \(Autoload\)](#)

Capturas del editor: documentación oficial de **Godot Engine** (Juan Linietsky, Ariel Manzur y la comunidad), bajo licencia **CC BY 4.0**.